

CS4432: Database Systems II

Static Hash Indexing

Updates

- Next Quiz (4) is on Thursday (**Thu Apr 16**)
- HW 2 Deadline **Extension due tomorrow (Tue Apr 14)**
- Project 2 is out
 - Deadline **Extension: Tue Apr 21**
- Now: **Reflection discussion**

Reflection Discussion

- Quicker grading of quizzes:
 - **Now it prompt**
 - **(+a lot of comments re quiz 1)**
- Also, quicker grading in general
- A lot of work
- Class sometime is going too slow (+boring?)
- Quicker responses
- More time for Quizzes (reference to 15 minutes)

Reflection Discussion

- Grades:
 - **Should be updated now**

Projects (3):	30%
Homeworks (3):	30%
Exam:	25%
Quizzes (3/4):	15%

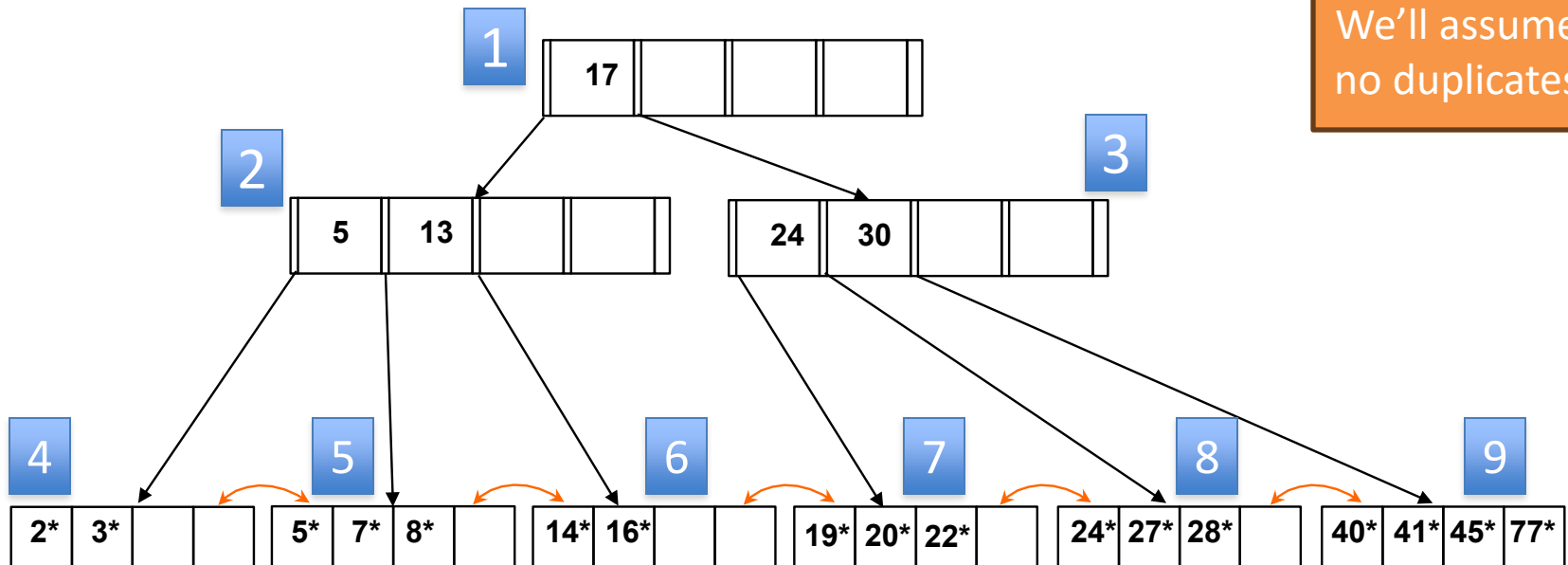
- **Quiz scores not visible?**

Reflection Discussion

- Clearer quiz questions? project instructions? Slides?
- Consistency between class and assignments?
- Quiz at the beginning of the class?
 - **Raise of hands**

B-Tree: Equality Search

We'll assume
no duplicates



Find key = 0, 2, 28, 101 ?

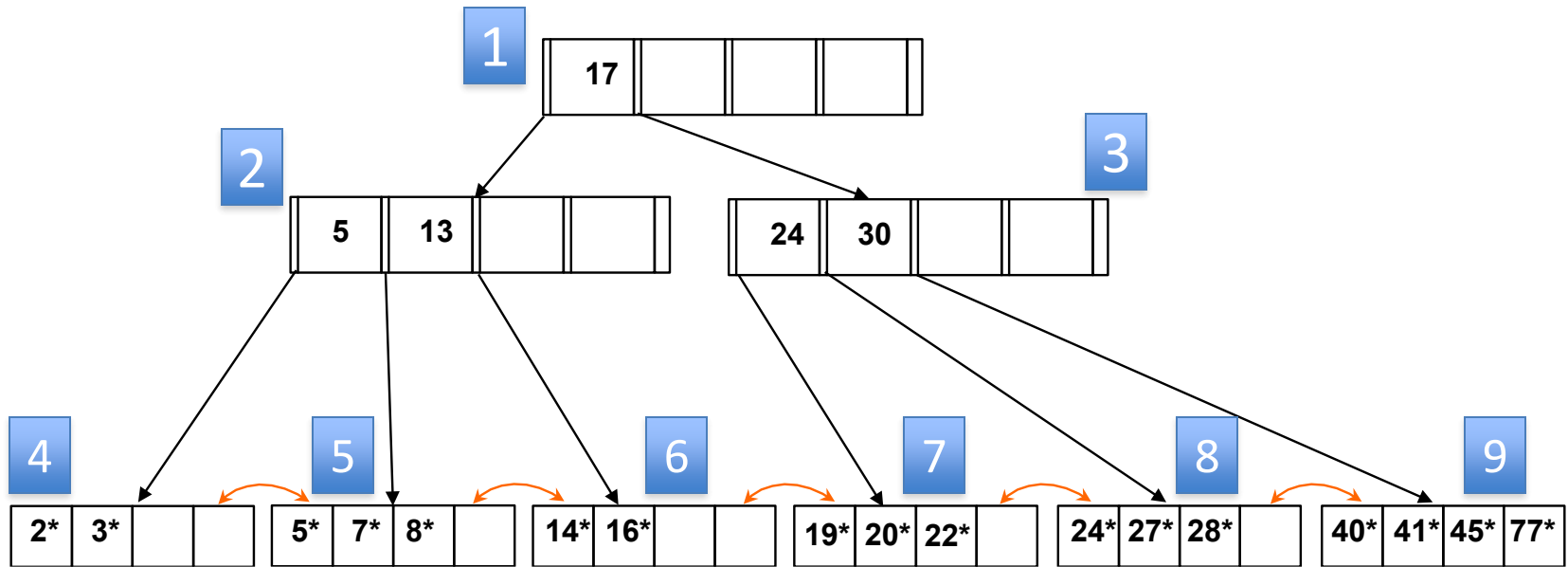
0 → N1, N2, N4

2 → N1, N2, N4, then read the data block

28 → N1, N3, N8, then read the data block

101 → N1, N3, N9

B-Tree: Range Search



Find key in [5, 20], [4, 16], [100, 200]?

[5, 20] → N1, N2, N5, N6, N7

[4, 16] → N1, N2, N4, N5, N6

[100, 200] → N1, N3, N9

Insertion Algorithm

- Find correct leaf **L**. (*Searching*)
- Put data entry onto **L**.
 - If **L** has enough space, *done!*
 - Else, must split **L** (into **L** and a new node **L2**)
 - Redistribute entries evenly, copy up middle key.
 - Insert index entry pointing to **L2** into parent of **L**.
- This can happen recursively
 - To split index node, redistribute entries evenly, but push up middle key. (Contrast with leaf splits.)
- Splits “grow” tree; root split increases height.
 - Tree growth: gets wider or one level taller at top.

Summary Insertion

- Practice the B-Tree insertions
- The split mechanism
 - Ensures each non-root node to be at least 50% full
 - Ensures the tree remains balanced
 - Grows the width of the tree
 - Root splitting increases the tree height

Deletion Algorithm

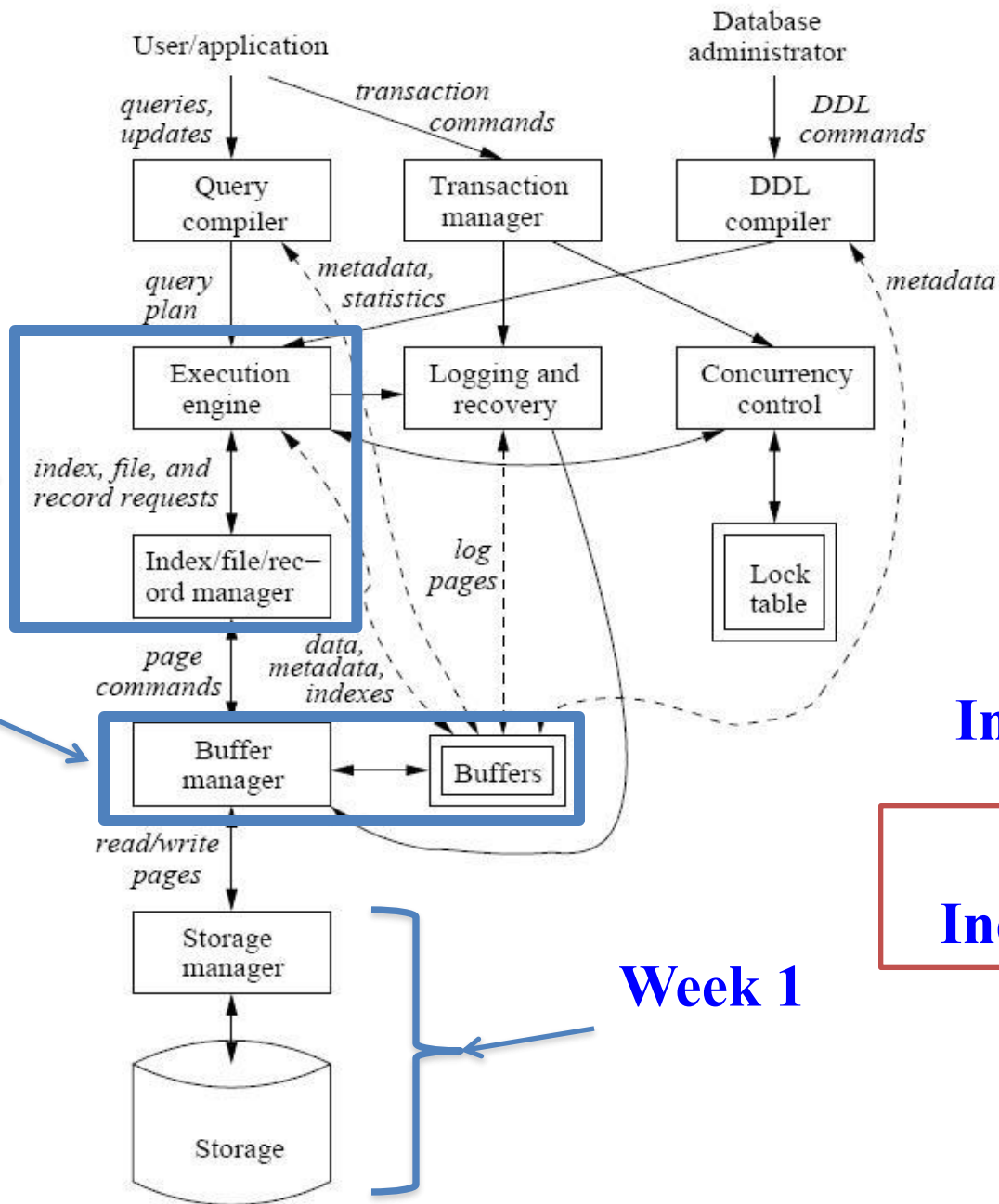
- Start at root, find **leaf** L where entry belongs. (*Searching*)
- Remove the entry.
 - If L is at least half-full, *done!* 😊
 - If L has less than **the minimum** entries,
 - Try to re-distribute, borrowing from sibling (*adjacent node with same parent as L*).
 - If re-distribution fails, merge L and sibling.
- If merge occurred, must delete entry (pointing to L or sibling) from parent of L .
- Merge could propagate to root, decreasing height.

Summary Deletion

- **Easy cases**
 - Key not found
 - Key is found and the leaf node > 50% full
- **Harder case**
 - Get help from either of your siblings (under the same parent)
 - Some key re-distribution
 - Deletion does not propagate upward
- **Hardest case**
 - Could not get help from siblings
 - Need to merge
 - Deletion propagates upward
- Borrowing from sibling (for internal node) ➔ Must involve the key from the parent

Week 3:
sorting +

Week 2



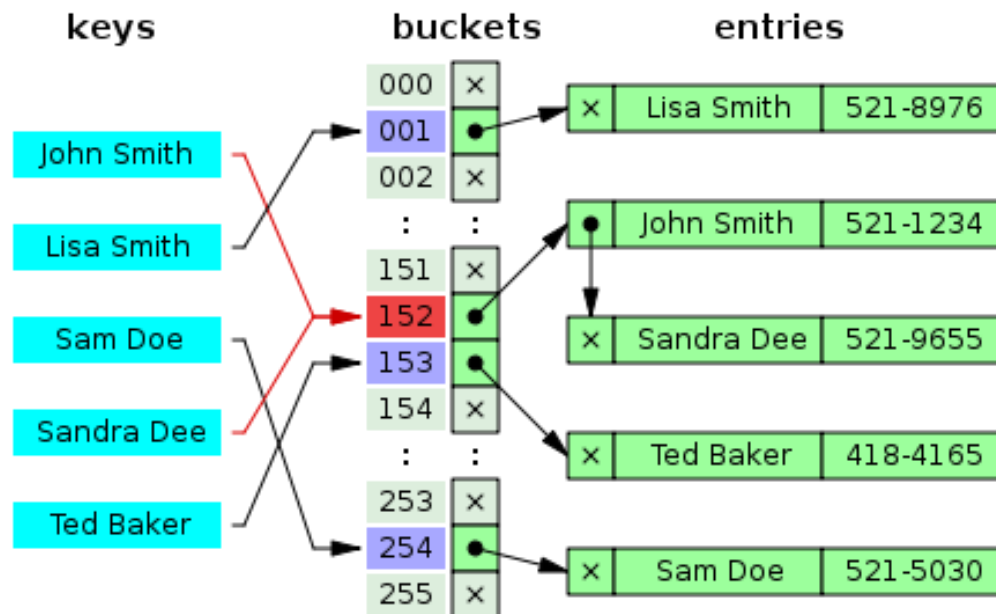
Database management system components

Learning objectives

- Understand differences of disk-based hashing
- Know how to implement static hashing:
 - Choice of hash function
 - Initialization
 - Insertion
 - Overflow buckets
 - Deletion
- Explain trade-offs involving choice of number of primary buckets N

Disk-Based Hash-Based Indexes

- Adaptation of main memory hash tables
- Support *equality* searches
- *No range searches*

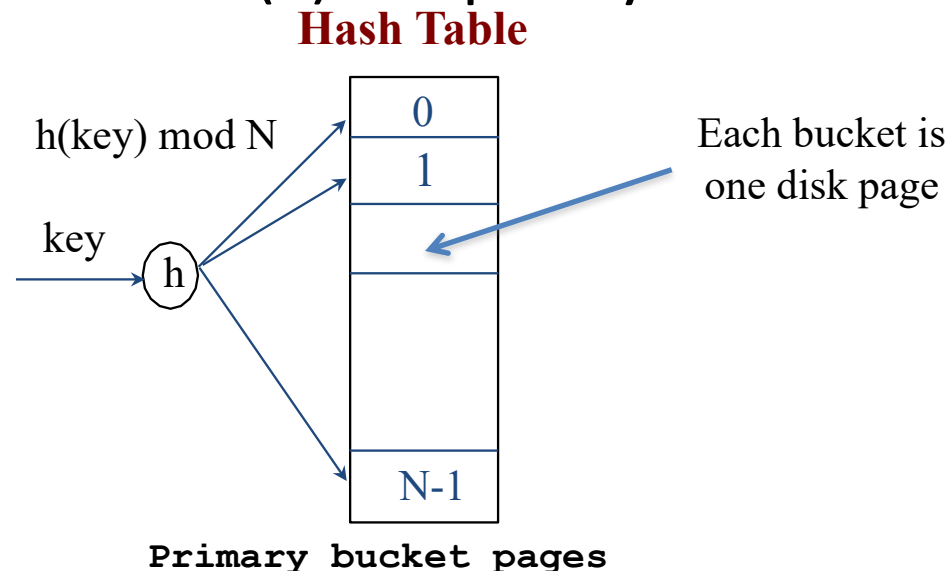


Disk-Based Hash-Based Indexes

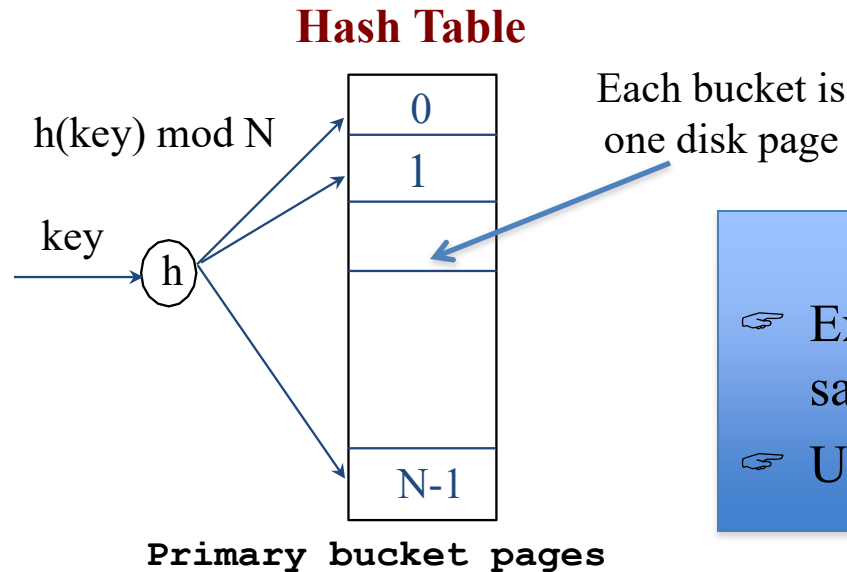
- **Static Hashing**
- **Extensible (Dynamic) Hashing**
 - After the break

Static Hashing

- Hash Table with **N buckets**
 - **(static means “N” does not change)**
- Since we talk about databases (disk-based)
 - Each bucket will be one disk page
- Hashing function $h(k)$ maps key k to one of the buckets



Some simple Hash Functions



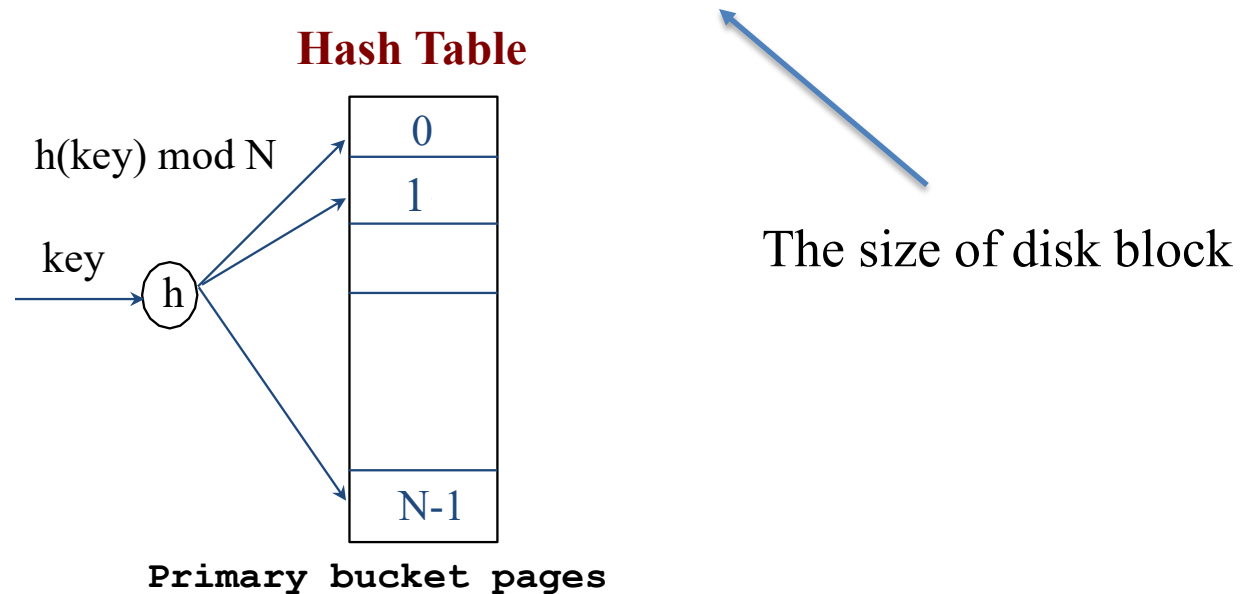
Good Hash Function

- ☞ Expected number of keys/bucket is the same for all buckets
- ☞ Uniform distribution of keys

- **If the key k is integer, e.g., 100**
 - Hash function: $k \bmod N$
- **If the key k is n -byte character string, e.g., “abcd”**
 - Hash function: $\text{add } (x_1 + x_2 + \dots + x_n) \bmod N$

Hash function must be deterministic

Within a Bucket



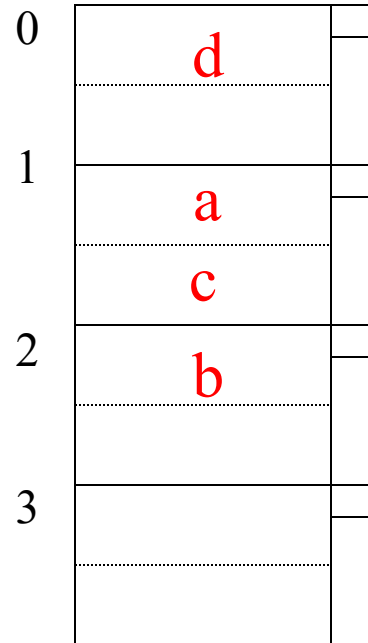
- **How a single entry in a bucket look like**

- Not only the key
- It has two components $\langle \text{key}, \text{RID} \rangle$

The key we are indexing

Record Id on disk in the user's table
($\text{rid} = \langle \text{block Id}, \text{slot\#} \rangle$)

Within a Bucket



- **Should we keep entries sorted?**
 - Yes, if we care about CPU time
 - Makes the insertion and deletion a bit more expensive

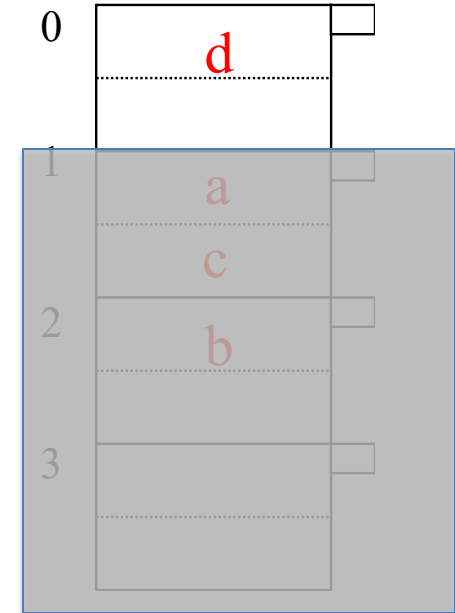
Hash Table: Lookup

Search for key = d



Remember: Only equality search

- 1- Apply the hash function over d $\rightarrow h(d) \bmod N = 0$
- 2- Read the disk page of bucket 0
- 3- Search for key d
 - If keys are sorted, then search using Binary search



Hash Table: Insertion

- We have $N=4$ buckets
- Each bucket holds 2 keys
- Insert keys a, b, c, and d

Do it yourself



INSERT:

$$h(a) \bmod N = 1$$

$$h(b) \bmod N = 2$$

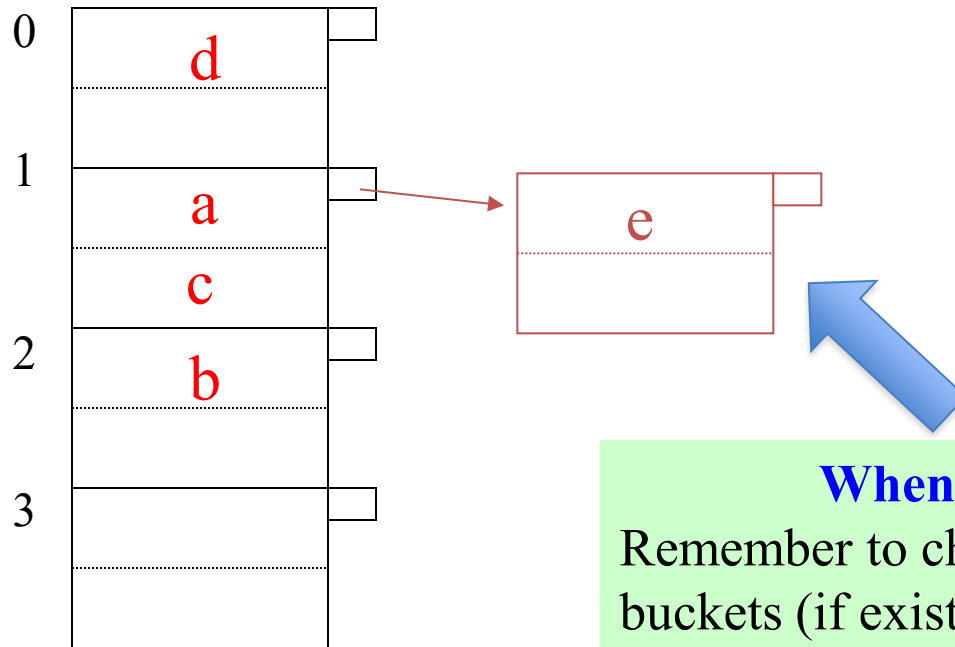
$$h(c) \bmod N = 1$$

$$h(d) \bmod N = 0$$

0	d	
1	a	
2	b	
3		

Hash Table: Insertion with Overflow

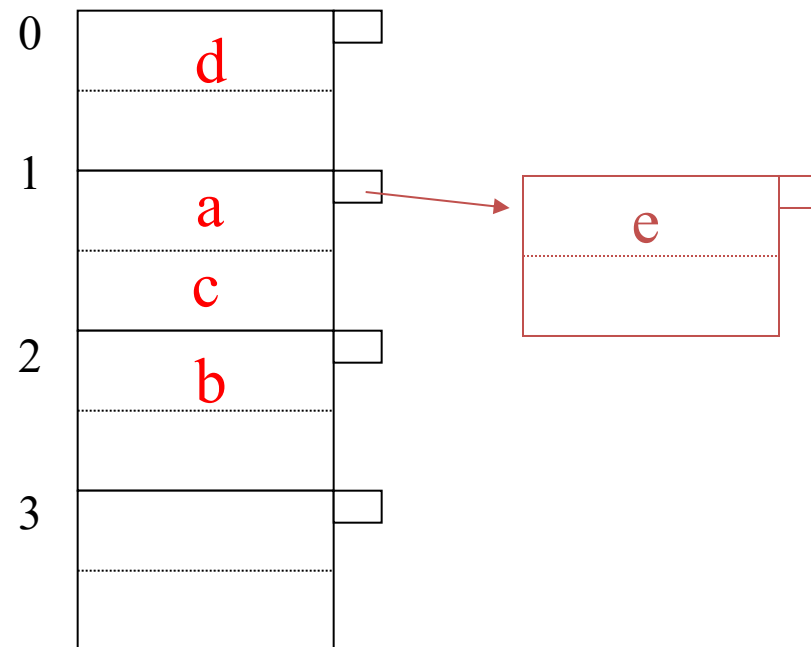
- Insert key $e \rightarrow h(e) \bmod N = 1$
- Create an **overflow bucket** and insert e
- Overflow bucket is another disk block



When Searching
Remember to check the overflow
buckets (if exist)

Hash Table: Deletion

- Search for the key to be deleted
- In case of overflow buckets
 - The overflow bucket may no longer be needed



Example: Deletion

Assume the following Hash Table

Do it yourself

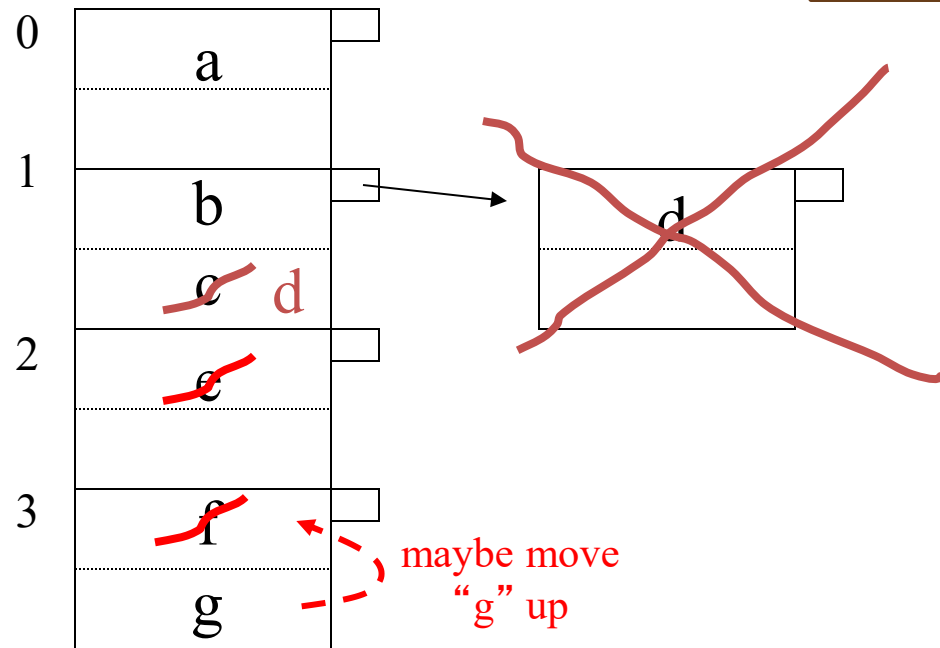


Delete:

e

f

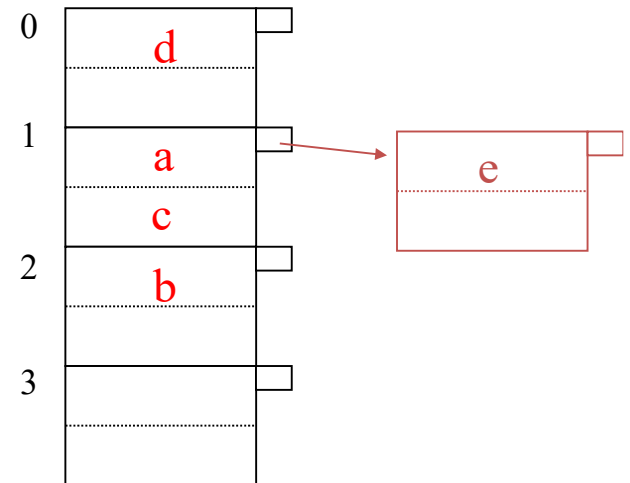
c



Summary of Static Hashing

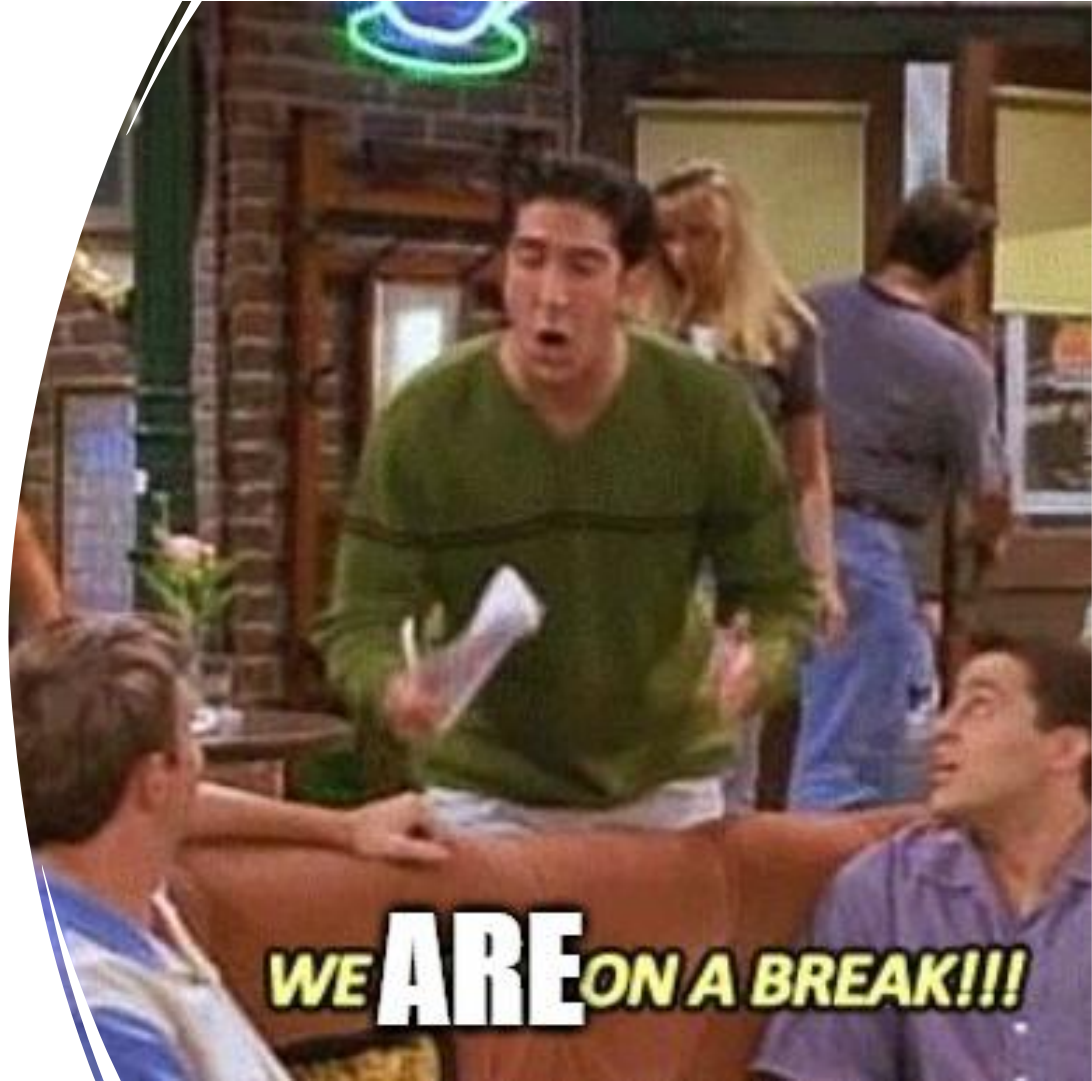
- In Static Hashing: **# primary buckets N is fixed**
- Common hash function is “mod”
- If there are many keys or key distribution is bad
 - Use overflow buckets
- **Bad News**
 - The chain of overflow buckets may get large
 - Search time become slow

Solution → Dynamic Hashing



Break

- Now we will take a few minutes break



CS4432: Database Systems II

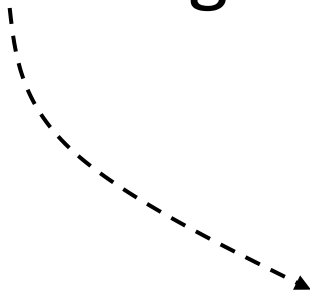
Dynamic Hash Indexing

Learning objectives

- Understand the concepts associated with Extensible Hashing: **directory**, **global** & **local depth**
- Implement insertion on Extensible Hash Index in two scenarios:
 - Local depth = global depth
 - Local depth < global depth
- Explain how it achieves I/O cost of 1

Dynamic Hashing

- No overflow buckets
- The number of primary buckets is not fixed and it can grow



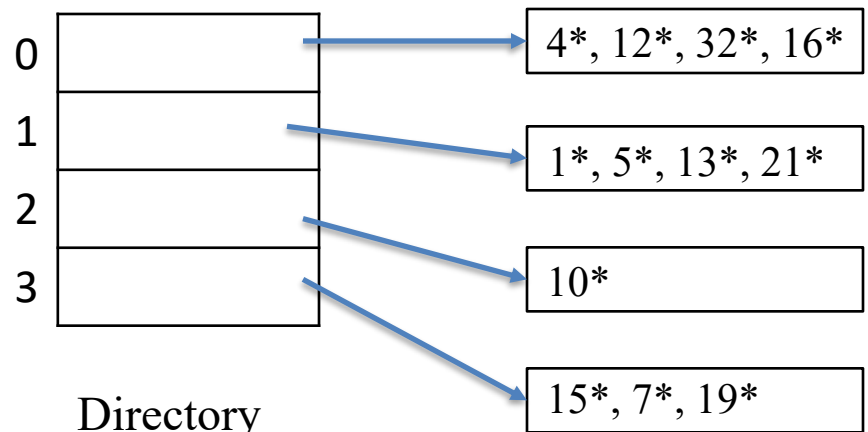
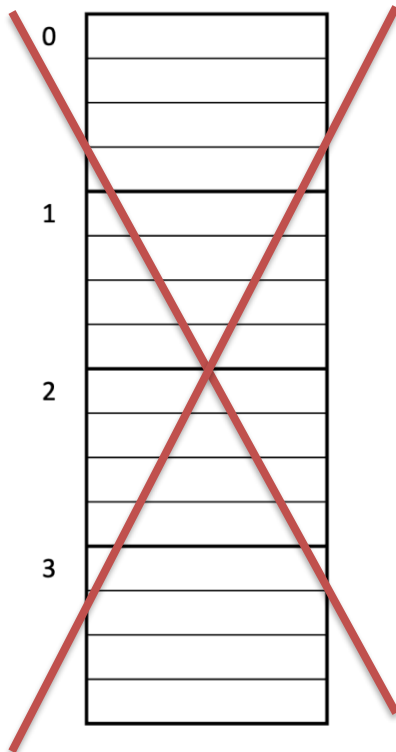
- Extensible Hashing
- Others ...



Our focus

Extensible Hash Index

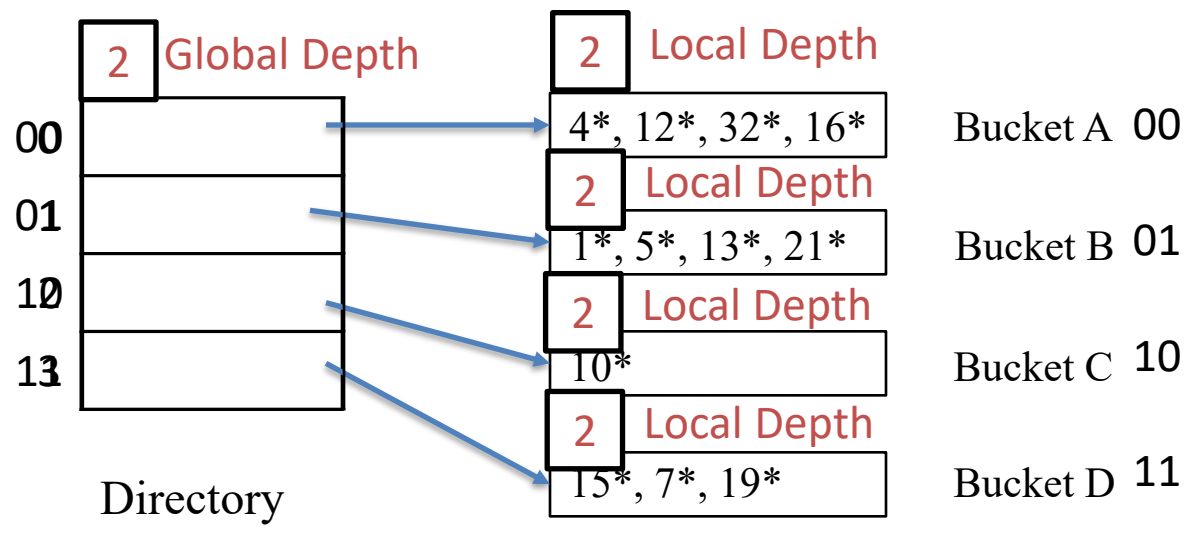
- Start with some # of buckets (e.g., $N=4$)
- Each bucket can fit fixed # of entries (e.g., $M=4$)
- **Problem:** storing index entries directly on primary bucket eventually requires overflow buckets
- **Solution:** Rather than storing entries directly, use a directory



What to do when
bucket becomes full?

Binary representation (N=4)

- Here we can use **2** bits to represent buckets (**global depth=2**)
- How can we find bucket for key 12? $[1100]_2$
 - Look at 2 rightmost bits of $h(12)$
- Every key in bucket A has the same 2 last bits (**local depth=2**)
 - Same for others



Extensible Hash Index: Hashing Function

- Given a key k → we need to know k will go to which entry in the directory
- **Proposed Hash Function**
 - Convert k into binary representation (0s and 1s)
 - Consider only the number of bits equal to the Global Depth value
 - Take bits from right side (Least Significant bits)

Extensible Hashing: Example

- Directory uses 2 bits (the right-most ones) → 4 entries
- Directory size = 4
- Each bucket holds at most 4 entries

Do it yourself



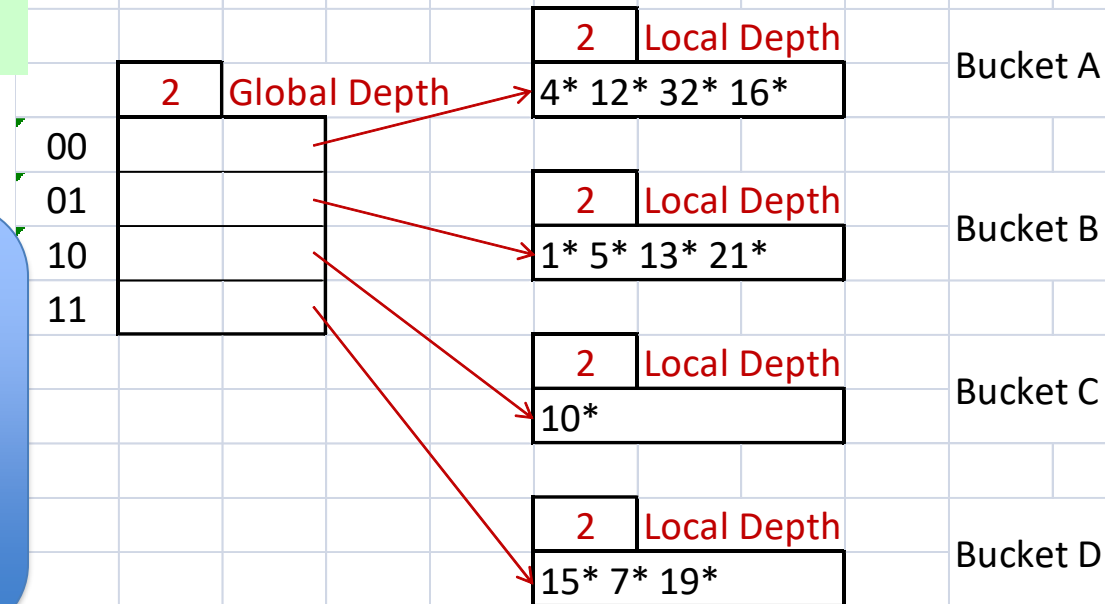
How did we
insert values
12, 10, 21?



	...	16	8	4	2	1
12→		1	1	0	0	
10→		1	0	1	0	
21→		1	0	1	0	1

The last two <global-depth> bits determine the bucket

In the beginning...



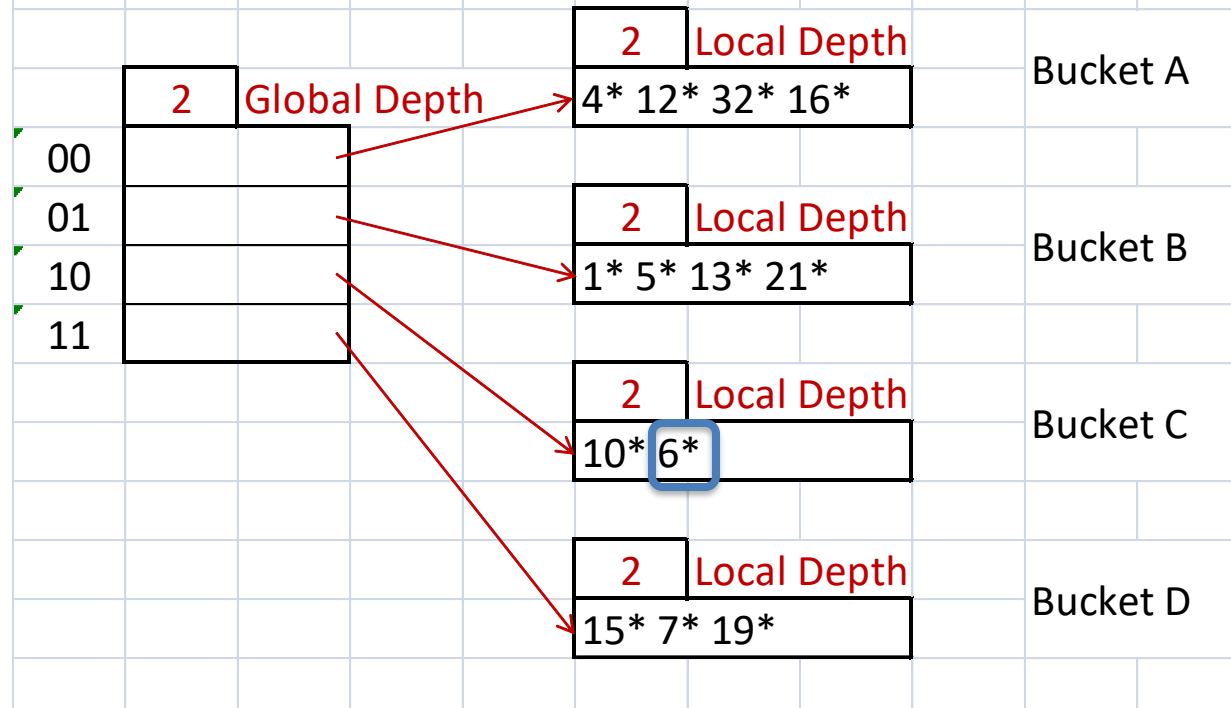
Inserting Key 6

Adding a value with $h(r) = 6$

Binary 6 = **110**, maps to Bucket C

Bucket C has room; just add it

Since global depth = 2,
we used only 2 right-
most bits



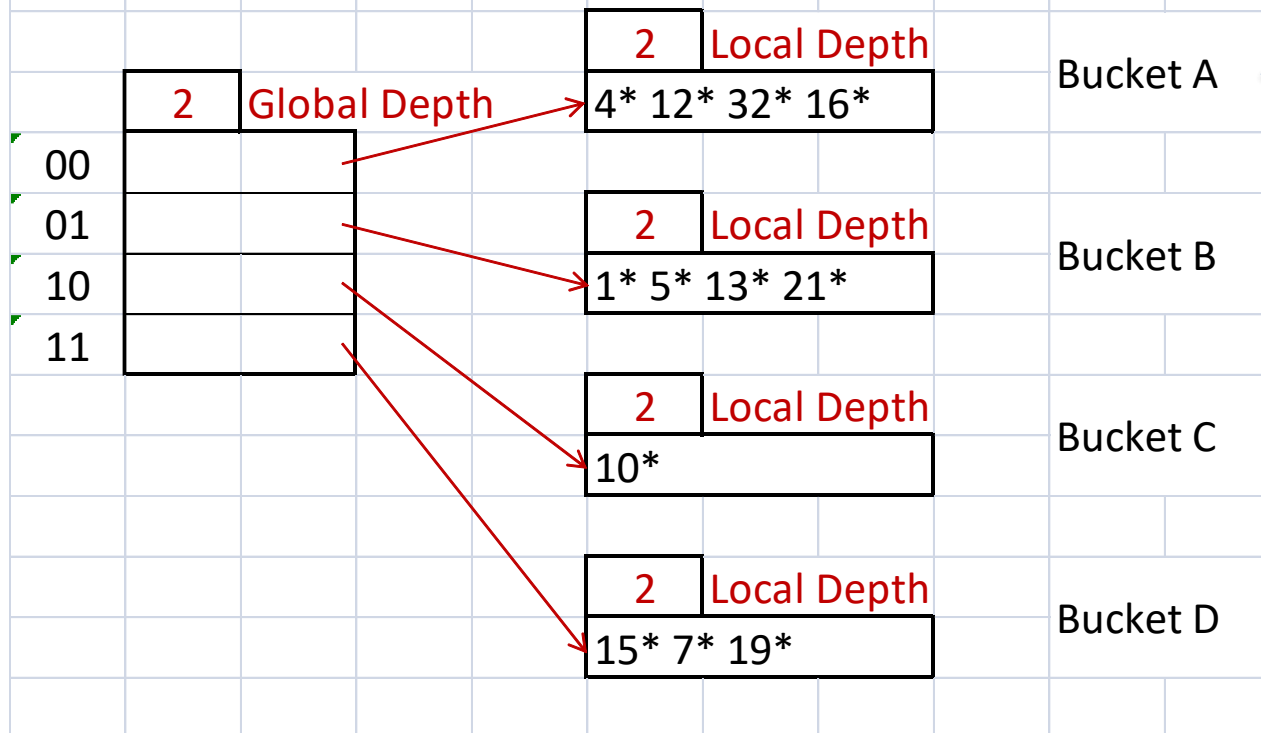
Inserting Key 20

Since global depth = 2,
we used only 2 right-
most bits

Adding a value with $h(r) = 20$

Binary 20 = 10100, maps to Bucket A

Bucket A has no room; what to do?



Bucket A is full:

-If local depth = global depth → double the size of Directory

Inserting Key 20

CASE 1: Expand under (Local Depth = Global Depth)

- 1- Increment global depth
- 2- This means → double the directory size

- 3- For the overflow bucket, divide into two
- 4- Increment their local depth
- 5- Re-distribute the keys *based on the new local depth bits*

- 6- For all other buckets, leave them as is (*local depth < global depth*)
- 7- Number of incoming pointers to each of these bucket is doubled

Adding a value with $h(r) = 20$

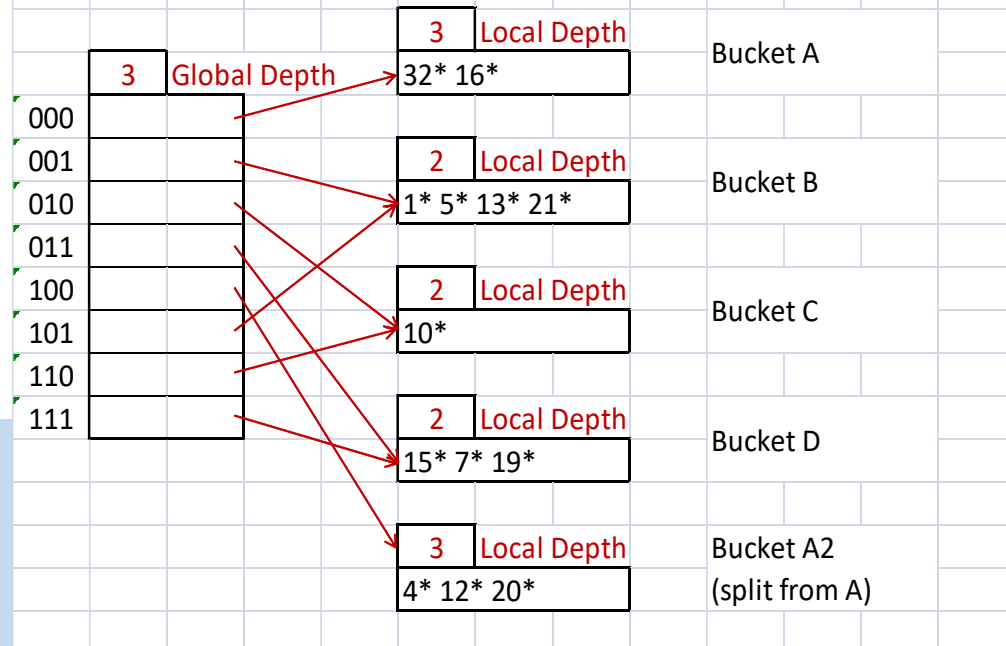
Binary 20 = 10100, maps to Bucket A

Bucket A has no room; what to do?

Split bucket A into two

The Directory needs to be split to accommodate A and A2

Happy ending



- For Buckets A & A2 → Keys are distributed based on 3 bits
- For Others → Keys are distributed based on 2 bits

Inserting Key 9

CASE 2: Split while Local Depth < Global Depth

- Key 9 → 1001 (global depth = 3)
- Key 9 → Bucket B (Full) ☹️
- Since local depth < global depth
 - No need to double Directory
 - Only split the bucket
 - Increment local depth
 - Re-distribute its keys

Adding a value with $h(r) = 20$

Binary 20 = 10100, maps to Bucket A

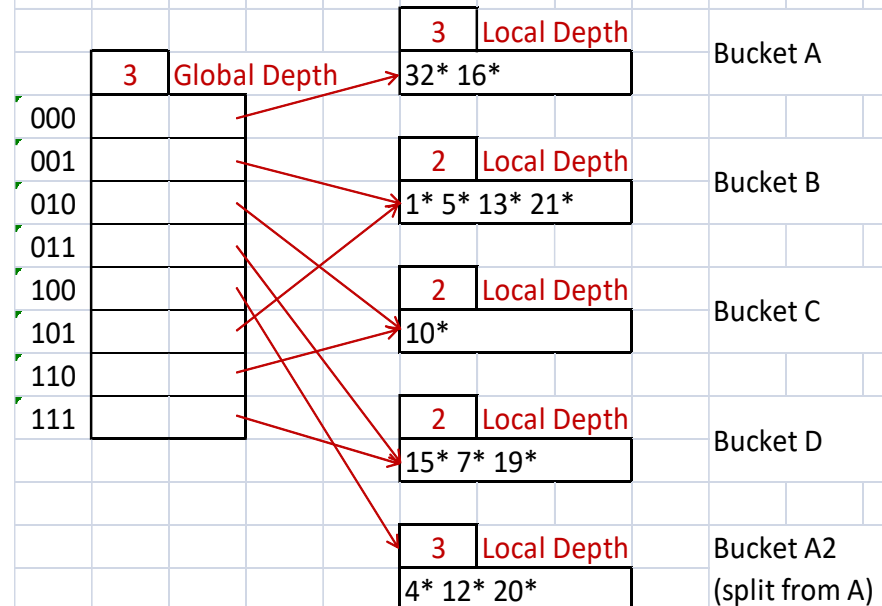
Bucket A has no room; what to do?

Split bucket A into two

The Directory needs to be split to accommodate A and A2

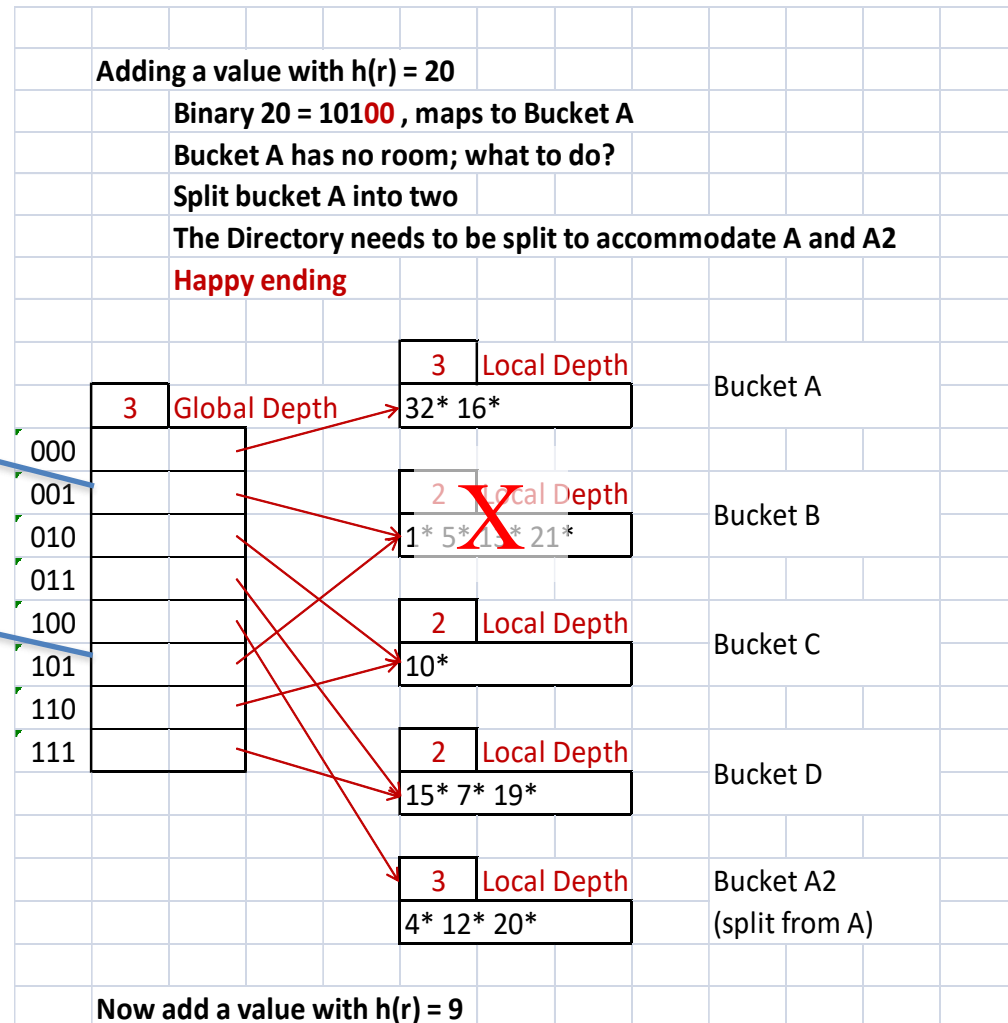
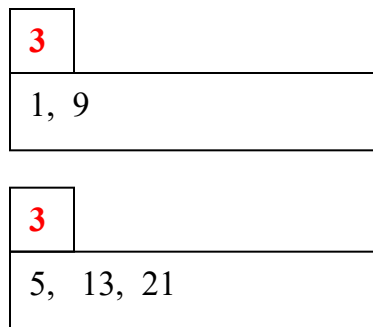
Happy ending

Do it yourself



Now add a value with $h(r) = 9$

Inserting Key 9



Extensible Hash Index Summary

- **Lookup:**
 - Global depth: # of bits needed to tell which bucket a datum belongs
 - Search the bucket
- **Insertion:**
 - If a bucket has room, insert the hash key
 - If no room,
 - May be able to add a new page without doubling (E.g., when adding 9*)
 - May need to double the directory (E.g., when adding 20*)
 - **How to tell if doubling is necessary?**
 - Doubling is necessary if $\text{Global Depth} = \text{Local Depth of overflow bucket}$